

LAB MANUAL

Machine Learning Practical Laboratory



Course Code: IT-593

Semester: 5th Semester

Prepared By:

Dinesh K. Pradhan, PhD

Instructor

Department of Information Technology

Dr. B. C. Roy Engineering College, Durgapur

Academic Year: 2026–27

Contents

List of Experiments	iii
Course Outcomes & Bloom's Mapping	v
General Lab Instructions	vi
Experiment 1: Python and ML Environment Basics	1
Experiment 2: Dataset Loading, Exploration, and Visualization	5
Experiment 3: Data Preprocessing	9
Experiment 4: Linear Regression	13
Experiment 5: Logistic Regression	17
Experiment 6: Decision Tree Classification	20
Experiment 7: KNN, Naive Bayes and SVM	23
Experiment 8: K-Means Clustering	26
Experiment 9: Hierarchical Clustering	29
Experiment 10: Principal Component Analysis	32
Experiment 11: Model Evaluation and Cross-Validation	35
Experiment 12: Ensemble Learning	39
Experiment 13: Hyperparameter Tuning using Grid Search	42
Experiment 14: Introduction to Neural Networks	46
Experiment 15: Mini-Project / Integrated Case Study	50
Extended Machine Learning Project Ideas	55
Laboratory Assessment Rubric	60
Recommended Python Environment Setup	61

List of Experiments

Machine Learning (IT-503) Lab – Experiments

Exp 1. Python and ML Environment Basics:

Set up the Python ML environment. Demonstrate Python data structures, NumPy arrays, Pandas DataFrames, and basic data manipulation.

Exp 2. Dataset Loading, Exploration, and Visualization:

Load a dataset using Pandas and perform Exploratory Data Analysis (EDA) utilizing descriptive statistics and Matplotlib/Seaborn visualizations.

Exp 3. Data Preprocessing:

Handle missing values, encode categorical variables, apply feature scaling (Standardization/Normalization), and perform train-test splitting.

Exp 4. Linear Regression:

Implement Simple and Multiple Linear Regression models. Evaluate prediction performance using standard regression metrics (e.g., MSE, RMSE, R-squared).

Exp 5. Logistic Regression:

Implement binary classification using Logistic Regression and evaluate the classifier using a confusion matrix and standard performance measures.

Exp 6. Decision Tree Classification:

Build a Decision Tree classifier, visualize the tree structure, and analyze the effect of tree depth (hyperparameter tuning) on classification accuracy.

Exp 7. K-Nearest Neighbours, Naive Bayes, and SVM:

Implement KNN, Naive Bayes, and Support Vector Machine (SVM) classifiers. Compare their predictive performance on a unified dataset.

Exp 8. K-Means Clustering:

Implement K-Means clustering, visualize the resulting clusters, and determine the optimal number of clusters using the Elbow method.

Exp 9. Hierarchical Clustering and Dendrograms:

Perform hierarchical clustering using agglomerative methods and visualize the hierarchical relationships using a dendrogram.

Exp 10. Principal Component Analysis (PCA):

Apply PCA for dimensionality reduction. Visualize high-dimensional data in a lower-dimensional space (2D/3D) while retaining variance.

Exp 11. Model Evaluation and Cross-Validation:

Implement k-fold cross-validation. Calculate and interpret comprehensive evaluation metrics including Accuracy, Precision, Recall, and F1-score.

Exp 12. Ensemble Learning (Random Forest & AdaBoost):

Implement Random Forest and AdaBoost classifiers. Compare their performance and robustness against a single Decision Tree model.

Exp 13. Hyperparameter Tuning using Grid Search:

Utilize `GridSearchCV` (and optionally `RandomizedSearchCV`) with cross-validation to identify optimal hyperparameters for a selected ML model.

Exp 14. Introduction to Neural Networks:

Implement a basic feed-forward neural network using a Multi-Layer Perceptron (MLP) classifier and evaluate its performance on a classification dataset.

Exp 15. Integrated Mini-Project / Case Study:

Develop an end-to-end machine learning application encompassing data preparation, model implementation, evaluation, comparative analysis, and final presentation.

Course Outcomes (CO) & Bloom's Mapping

Course Learning Outcomes

CO	Description	Bloom's Level
CO1	Understand the fundamental concepts, types, workflow, and applications of Machine Learning.	L2 (Understand)
CO2	Apply data preprocessing, supervised and unsupervised learning techniques to solve machine learning problems.	L3 (Apply)
CO3	Implement and evaluate machine learning models using suitable datasets, performance metrics, and validation methods.	L4 (Analyze)
CO4	Design and develop comparative machine learning solutions using ensemble methods, hyperparameter tuning, and introductory neural networks.	L6 (Create)

Recommended Software Environment

- Python 3.x
- Jupyter Notebook / JupyterLab / Google Colab
- NumPy
- Pandas
- Matplotlib
- Seaborn
- Scikit-learn
- Mlxtend (for Association Rule extension, when required)
- Optional: VS Code / PyCharm

General Lab Instructions

Guidelines for Students

- **Pre-lab Preparation:** Study the relevant Machine Learning theory and mathematical concepts prior to attending the laboratory session.
- **Lab Execution:** All experiments must be independently implemented, debugged, and executed during the scheduled laboratory hours.
- **Coding Standards:** Write clean, professional code. Maintain proper Python indentation, write meaningful variable names, and include explanatory inline comments.
- **Lab Record Structure:** Every documented experiment must systematically include: Objective, Theory, Algorithm/Steps, Source Code, Output/Visualizations, Result/Inference, Viva Questions, and Practice Problems.
- **Conceptual Clarity:** Focus on understanding the dataset characteristics and the inner workings of the algorithm rather than merely executing the code mechanically.
- **Model Justification:** The choice of a specific machine learning algorithm should always be logically justified based on the problem statement and data type.
- **Performance Tracking:** Rigorously record and analyze model performance using appropriate evaluation metrics (e.g., Accuracy, MSE, F1-Score).
- **Data Visualization:** Ensure all graphs, plots, dendrograms, and confusion matrices are clearly legible and properly labelled (including titles, axis labels, and legends).
- **Independent Experimentation:** Students are highly encouraged to experiment with different hyperparameters (e.g., learning rates, tree depths, k -values) and document their effects on model performance.
- **Assessments & Viva:** Be prepared for regular viva-voce evaluations, which will encompass theoretical concepts, application-based scenarios, and code-comprehension questions.
- **Submission Verification:** Lab records must be submitted on time. Include clear screenshots or equivalent visual evidence of successful code execution and outputs wherever required.

Important: Machine Learning experiments should emphasize interpretation of results, not only successful execution of code.

Experiment 1: Python and ML Environment Basics

Objective

- Understand the fundamental Python environment required for Machine Learning.
- Demonstrate the creation of NumPy arrays and perform basic numerical operations.
- Create, manipulate, and analyze tabular data using Pandas Series and DataFrames.
- Perform rudimentary statistical analysis and exploratory data checks.

Theory

- **NumPy (Numerical Python):** The core library for scientific computing in Python. It provides high-performance multidimensional array objects (`ndarray`) and tools for working with these arrays, which are essential for matrix operations in ML.
- **Pandas:** An open-source data analysis and manipulation tool built on top of Python. It provides highly optimized performance with back-end source code purely written in C or Python.
- **DataFrame:** A two-dimensional, size-mutable, and potentially heterogeneous tabular data structure with labeled axes (rows and columns) in Pandas.
- **Exploratory Analysis Methods:** Functions such as `head()`, `info()`, and `describe()` are used to quickly inspect data types, detect missing values, and calculate summary statistics.

Course Outcome (CO) Mapping

CO1	CO2	CO3
-----	-----	-----

Algorithm / Steps

Step 1. Import the required libraries: `numpy` and `pandas`.

Step 2. Initialize a NumPy array with sample numerical data and perform aggregate mathematical operations (mean, max, min).

Step 3. Construct a Python dictionary containing student information (Name, Attendance, Marks).

Step 4. Convert the dictionary into a Pandas DataFrame.

Step 5. Display the first few rows of the DataFrame using the `head()` function.

Step 6. Extract structural information and statistical summaries using `info()` and `describe()`.

Step 7. Isolate a specific column (e.g., Marks) and compute its average.

Python Program

Source Code

```
import numpy as np
import pandas as pd

# 1. NumPy Array Operations
marks = np.array([72, 85, 91, 68, 77])

print("Marks:", marks)
print("Mean:", np.mean(marks))
print("Maximum:", np.max(marks))
print("Minimum:", np.min(marks))

# 2. Pandas DataFrame Creation
data = {
    "Name": ["Amit", "Riya", "Sourav", "Neha", "Rahul"],
    "Attendance": [88, 92, 76, 95, 81],
    "Marks": [72, 85, 68, 91, 77]
}

df = pd.DataFrame(data)

# 3. Data Exploration
print("\n--- First Five Records ---")
print(df.head())

print("\n--- Data Information ---")
print(df.info())

print("\n--- Statistical Summary ---")
print(df.describe())

print("\nAverage Marks:", df["Marks"].mean())
```

Sample Output

Output

```
Marks: [72 85 91 68 77]
Mean: 78.6
Maximum: 91
Minimum: 68

--- First Five Records ---
   Name  Attendance  Marks
0  Amit           88     72
1  Riya           92     85
2  Sourav          76     68
3  Neha           95     91
4  Rahul           81     77

--- Data Information ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  -
0  Name         5 non-null      object
1  Attendance    5 non-null      int64
2  Marks         5 non-null      int64
dtypes: int64(2), object(1)
memory usage: 248.0+ bytes
None

--- Statistical Summary ---
   Attendance  Marks
count  5.000000  5.000000
mean   86.400000 78.600000
std    7.700649  9.289779
min    76.000000 68.000000
25%    81.000000 72.000000
50%    88.000000 77.000000
75%    92.000000 85.000000
max    95.000000 91.000000

Average Marks: 78.6
```

Result / Conclusion

The Python environment was successfully configured. Basic data structures including NumPy arrays and Pandas DataFrames were created, and fundamental data analysis operations required for Machine Learning workflows were executed without errors.

Viva Questions

- Q1.** What is the core difference between a standard Python list and a NumPy array in terms of memory and performance?
- Q2.** How does a Pandas DataFrame differ from a Pandas Series?
- Q3.** What specific information does the `df.info()` method display, and why is it useful before training an ML model?
- Q4.** Why is Pandas universally preferred over standard spreadsheet software (like Excel) in Machine Learning?
- Q5.** What statistics are generated by default when `df.describe()` is called on numerical columns?

Assignment / Practice Problems

1. Create a NumPy array containing the internal marks of 10 students. Calculate and print the mean, median, standard deviation, maximum, and minimum values.
2. Construct a DataFrame containing the following columns: **Student_Name**, **Roll_Number**, **Marks**, and **Attendance**. Filter and display only the records of students who have scored above 80 marks.
3. Write a script to add a new dynamically calculated column named **Grade** to the DataFrame based on the **Marks** column (e.g., ≥ 90 is 'A', 80-89 is 'B', etc.).

Experiment 2: Dataset Loading, Exploration, and Visualization

Objective

- Load a standard Machine Learning dataset using Pandas and Scikit-learn.
- Perform Exploratory Data Analysis (EDA) to understand data distributions.
- Identify and differentiate between numerical and categorical variables.
- Visualize important feature relationships using Matplotlib and Seaborn.

Theory

Exploratory Data Analysis (EDA) is the critical first step of examining a dataset before building any machine learning model. Its primary goal is to discover patterns, spot anomalies, test hypotheses, and check assumptions with the help of summary statistics and graphical representations.

Important EDA operations include:

- **Dataset Inspection:** Understanding the dimensions, data types, and missing values.
- **Descriptive Statistics:** Calculating mean, median, standard deviation, and quartiles.
- **Distribution Analysis:** Checking if features are normally distributed or skewed.
- **Correlation Analysis:** Quantifying the linear relationship between different variables.
- **Visualization:** Using scatter plots, histograms, and boxplots to visually reveal patterns, outliers, and data quality issues.

Course Outcome (CO) Mapping

CO1	CO2	CO3
-----	-----	-----

Algorithm / Steps

Step 1. Import essential libraries: `pandas`, `matplotlib.pyplot`, and `seaborn`.

Step 2. Load the Iris dataset using `sklearn.datasets`.

Step 3. Construct a Pandas DataFrame using the dataset features and append the target labels.

Step 4. Display the top rows and generate descriptive statistical summaries.

Step 5. Check the dataset for any missing (null) values.

Step 6. Compute and display the correlation matrix for numerical features.

Step 7. Plot a Seaborn scatter plot to visualize the relationship between two specific features.

Step 8. Plot a histogram (with KDE) to observe the distribution of a single feature.

Python Program

Source Code

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris

# 1. Load dataset
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df["target"] = iris.target

# 2. Basic Data Exploration
print("--- First Five Rows ---")
print(df.head())

print("\n--- Dataset Information ---")
print(df.info())

print("\n--- Statistical Summary ---")
print(df.describe())

print("\n--- Missing Values ---")
print(df.isnull().sum())

# 3. Correlation matrix
print("\n--- Correlation Matrix ---")
print(df.corr(numeric_only=True))

# 4. Scatter plot: Sepal Length vs Petal Length
plt.figure(figsize=(7, 5))
sns.scatterplot(
    data=df,
    x="sepal length (cm)",
    y="petal length (cm)",
    hue="target",
    palette="viridis"
)
```

```
plt.title("Sepal Length vs Petal Length by Class")
plt.show()

# 5. Histogram: Distribution of Sepal Length
plt.figure(figsize=(7, 5))
sns.histplot(df["sepal length (cm)"], kde=True, color="blue")
plt.title("Distribution of Sepal Length")
plt.show()
```

Sample Output

Console Output

```
--- First Five Rows ---
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  target
0                5.1                3.5                1.4                0.2         0
1                4.9                3.0                1.4                0.2         0
2                4.7                3.2                1.3                0.2         0
3                4.6                3.1                1.5                0.2         0
4                5.0                3.6                1.4                0.2         0

--- Dataset Information ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
...
dtypes: float64(4), int64(1)

--- Missing Values ---
sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
target              0
dtype: int64

--- Correlation Matrix ---
               sepal length (cm)  sepal width (cm)  ...    target
sepal length (cm)             1.000000        -0.117570  ...    0.782561
sepal width (cm)             -0.117570         1.000000  ...   -0.426658
petal length (cm)             0.871754        -0.428440  ...    0.949035
...
```

Note: Graphical outputs (Scatter Plot and Histogram) will render in separate GUI windows or inline if using Jupyter Notebook. Students should attach screenshots of their generated plots here.

Result / Conclusion

The dataset was successfully loaded, structurally explored, and statistically summarized. Important data distributions and feature relationships were visually examined using graphical techniques (histograms and scatter plots), confirming the dataset is ready for preprocessing and model training.

Viva Questions

- Q1.** What is the primary objective of Exploratory Data Analysis (EDA)?
- Q2.** Why is data visualization a critical step before beginning the Machine Learning modeling phase?
- Q3.** What does a correlation matrix tell us, and what does a value close to 1 or -1 indicate?
- Q4.** What specific data characteristics does a histogram combined with a KDE (Kernel Density Estimate) curve show?
- Q5.** What is a statistical outlier, and which visualization plot is typically best for identifying them?

Assignment / Practice Problems

1. Load and perform a complete EDA on the **Wine** dataset available in Scikit-learn.
2. Plot Boxplots for all numerical attributes in the Wine dataset to visually identify any potential outliers.
3. Generate a correlation heatmap (using Seaborn's `heatmap`) and identify the pair of features that exhibit the strongest positive correlation.

Experiment 3: Data Preprocessing

Objective

- Handle missing values in numerical and categorical data.
- Encode categorical variables into numerical format.
- Perform feature scaling to standardize the range of independent variables.
- Split the dataset into training and testing subsets for model evaluation.

Theory

Machine learning models require properly formatted and cleaned data to achieve high accuracy and prevent erroneous predictions.

Common preprocessing operations include:

- **Missing-Value Treatment:** Using techniques like mean, median, or mode imputation to fill gaps in data.
- **Categorical Encoding:** Converting text labels into numbers (e.g., One-Hot Encoding, Label Encoding) so algorithms can process them.
- **Feature Scaling:** Ensuring all numerical features contribute equally to the distance calculations in algorithms.
- **Data Splitting:** Dividing data into a training set (to train the model) and a testing set (to evaluate generalization).

A widely used feature scaling method is **Standardization** (Z-score normalization), which centers the data around a mean of zero with a standard deviation of one:

$$z = \frac{x - \mu}{\sigma}$$

where μ is the mean of the feature and σ is the standard deviation.

Course Outcome (CO) Mapping

CO1	CO2	CO3
-----	-----	-----

Algorithm / Steps

Step 1. Create a mock dataset containing both numerical and categorical attributes, explicitly introducing missing values (NaN).

Step 2. Separate the dataset into independent features (X) and the dependent target variable (y).

- Step 3.** Define data preprocessing pipelines for numeric and categorical features separately.
- Step 4.** Apply `SimpleImputer` to replace missing numerical values using the median strategy.
- Step 5.** Encode categorical variables using `OneHotEncoder` and impute missing text data using the most frequent strategy.
- Step 6.** Standardize the numerical features using `StandardScaler`.
- Step 7.** Combine the transformations using a `ColumnTransformer`.
- Step 8.** Split the processed dataset into training and testing subsets using `train_test_split`.

Python Program

Source Code

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline

# 1. Create Mock Dataset
data = {
    "Age": [20, 21, None, 23, 24],
    "Income": [25000, 30000, 28000, None, 40000],
    "City": ["Kolkata", "Durgapur", "Kolkata", "Asansol", "Durgapur"],
    "Purchased": [0, 1, 1, 0, 1]
}

df = pd.DataFrame(data)

# Separate Features and Target
X = df.drop("Purchased", axis=1)
y = df["Purchased"]

numeric_features = ["Age", "Income"]
categorical_features = ["City"]

# 2. Define Preprocessing Pipelines
numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])
```

```
] )

# 3. Combine using ColumnTransformer
preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numeric_features),
        ("cat", categorical_transformer, categorical_features)
    ]
)

# Apply Transformations
X_processed = preprocessor.fit_transform(X)

# 4. Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X_processed, y, test_size=0.2, random_state=42
)

# Display Results
print("--- Original Dataset ---")
print(df)

print("\n--- Processed Feature Matrix Shape ---")
print(X_processed.shape)

print("\nTraining Samples:", X_train.shape[0])
print("Testing Samples:", X_test.shape[0])
```

Sample Output

Console Output

```
--- Original Dataset ---
   Age  Income  City  Purchased
0  20.0  25000.0  Kolkata         0
1  21.0  30000.0  Durgapur         1
2   NaN  28000.0  Kolkata         1
3  23.0     NaN  Asansol         0
4  24.0  40000.0  Durgapur         1

--- Processed Feature Matrix Shape ---
(5, 5)

Training Samples: 4
Testing Samples: 1
```

Result / Conclusion

The dataset was successfully preprocessed by handling missing values (imputation), converting categorical strings to numerical format (One-Hot Encoding), scaling the continuous variables (Standardization), and dividing the final transformed data into mutually exclusive training and testing sets.

Viva Questions

- Q1.** Why is data preprocessing a mandatory step before training most Machine Learning algorithms?
- Q2.** What is the core mathematical and conceptual difference between Normalization (Min-Max Scaling) and Standardization (Z-score)?
- Q3.** Why must categorical variables (like city names) be encoded rather than passed directly to ML models?
- Q4.** Why should preprocessing functions (like standard scaler) be fitted *only* on the training data rather than the entire dataset?
- Q5.** What is "data leakage" in Machine Learning, and how does proper train-test splitting prevent it?

Assignment / Practice Problems

1. Create and preprocess a synthetic dataset containing the features: **Age**, **Salary**, **Department**, and **Years_of_Experience**. Inject missing values deliberately.
2. Modify the pipeline code to apply **MinMaxScaler** instead of **StandardScaler**.
3. Compare and document the effect of **StandardScaler** vs **MinMaxScaler** by printing the transformed arrays and observing their numerical ranges.

Experiment 4: Linear Regression

Objective

- Implement Simple Linear Regression on a 2D dataset.
- Build and train a predictive regression model using Scikit-learn.
- Evaluate the model's performance using standard regression metrics.
- Visualize the actual data points alongside the predicted regression line.

Theory

Linear Regression models the linear relationship between a continuous independent (input) variable and a continuous dependent (target) variable.

For simple linear regression, the hypothesis equation is:

$$\hat{y} = b_0 + b_1x$$

where:

- $\hat{y}$ = Predicted output (dependent variable)
- x = Input feature (independent variable)
- b_1 = Slope (weight/coefficient of the input)
- b_0 = y-intercept (bias)

Course Outcome (CO) Mapping

CO1	CO2	CO3
-----	-----	-----

Algorithm / Steps

Step 1. Create a synthetic dataset mapping study hours to exam scores.

Step 2. Split the dataset into training and testing subsets (e.g., 75% train, 25% test).

Step 3. Initialize a `LinearRegression` model.

Step 4. Train the model using the `fit()` function on the training set.

Step 5. Generate predictions on the testing set using the `predict()` function.

Step 6. Calculate evaluation metrics: Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and the R^2 Score.

Step 7. Plot the original scatter data and overlay the fitted regression line.

Python Program

Source Code

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# 1. Dataset Creation
X = np.array([1, 2, 3, 4, 5, 6, 7, 8]).reshape(-1, 1)
y = np.array([35, 40, 50, 55, 60, 68, 75, 82])

# 2. Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

# 3. Model Initialization & Training
model = LinearRegression()
model.fit(X_train, y_train)

# 4. Prediction
y_pred = model.predict(X_test)

# 5. Metrics Calculation
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("Slope (b1):", model.coef_[0])
print("Intercept (b0):", model.intercept_)
print("\n--- Evaluation Metrics ---")
print(f"MAE : {mae:.2f}")
print(f"MSE : {mse:.2f}")
print(f"RMSE : {rmse:.2f}")
print(f"R2 : {r2:.4f}")

# 6. Plotting
plt.figure(figsize=(7, 5))
plt.scatter(X, y, color="blue", label="Actual Data")
plt.plot(X, model.predict(X), color="red", linewidth=2, label="Regression Line")
plt.xlabel("Study Hours")
plt.ylabel("Exam Score")
```

```
plt.title("Study Hours vs Score")
plt.legend()
plt.grid(True, linestyle="--", alpha=0.6)
plt.show()
```

Sample Output

Console Output

```
Slope (b1): 6.857142857142858
Intercept (b0): 27.28571428571428
```

```
--- Evaluation Metrics ---
```

```
MAE   : 1.57
MSE   : 3.33
RMSE  : 1.82
R2    : 0.9813
```

Note: The Matplotlib graph showing the scatter plot and the red regression line will appear in a separate window. Students should attach a screenshot of the graph here.

Result / Conclusion

The Simple Linear Regression model was successfully trained and evaluated. The metrics confirm a strong linear relationship between study hours and exam scores, visually corroborated by the regression line accurately fitting the scatter data.

Viva Questions

- Q1.** What is the conceptual difference between correlation and linear regression?
- Q2.** What does the R^2 (Coefficient of Determination) score intuitively indicate about a model?
- Q3.** In the equation of a line, what physical or rate meaning does the slope (b_1) represent?
- Q4.** What happens to a linear regression model's performance if the underlying data relationship is highly non-linear?
- Q5.** Why do we use Mean Squared Error (MSE) instead of just Mean Absolute Error (MAE) when calculating loss?

Assignment / Practice Problems

1. Download a real-estate dataset and predict house prices based solely on the house area (in sq. ft).
2. Implement Multiple Linear Regression using two or more features (e.g., house area and number of bedrooms).
3. Implement Polynomial Regression on the same dataset and compare the R^2 score against standard Linear Regression.

Experiment 5: Logistic Regression

Objective

- Implement binary classification using Logistic Regression.
- Understand how logistic models map inputs to probability-based predictions.
- Evaluate classification performance using Accuracy, Precision, Recall, and F1-score.
- Visualize the decision boundary or classification results where appropriate.

Theory

Despite its name, Logistic Regression is a *classification* algorithm used to estimate the probability that an instance belongs to a particular class.

It passes a linear equation through the **Sigmoid (Logistic) Function**, which bounds the output between 0 and 1:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

where $z = b_0 + b_1x_1 + \dots + b_nx_n$.

The probability output is converted into a discrete class label using a decision threshold (typically 0.5):

- If $P(y = 1|X) \geq 0.5$, classify as Class 1.
- If $P(y = 1|X) < 0.5$, classify as Class 0.

Course Outcome (CO) Mapping

CO1	CO2	CO3
-----	-----	-----

Algorithm / Steps

Step 1. Load the standard Breast Cancer dataset from Scikit-learn.

Step 2. Split the dataset into training and testing sets, ensuring class stratification.

Step 3. Apply `StandardScaler` to normalize the feature scales.

Step 4. Initialize and train a `LogisticRegression` classifier.

Step 5. Predict the discrete class labels for the testing set.

Step 6. Calculate classification evaluation metrics: Accuracy, Precision, Recall, and F1-score.

Python Program

Source Code

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score,
    f1_score

# 1. Load Data
data = load_breast_cancer()
X = data.data
y = data.target

# 2. Train-Test Split (Stratified)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 3. Feature Scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 4. Model Training
model = LogisticRegression(max_iter=1000, random_state=42)
model.fit(X_train, y_train)

# 5. Prediction
y_pred = model.predict(X_test)

# 6. Evaluation Metrics
print("--- Logistic Regression Performance ---")
print(f"Accuracy : {accuracy_score(y_test, y_pred):.4f}")
print(f"Precision : {precision_score(y_test, y_pred):.4f}")
print(f"Recall : {recall_score(y_test, y_pred):.4f}")
print(f"F1 Score : {f1_score(y_test, y_pred):.4f}")
```

Sample Output

Console Output

```
--- Logistic Regression Performance ---  
Accuracy   : 0.9737  
Precision   : 0.9726  
Recall      : 0.9861  
F1 Score    : 0.9793
```

Result / Conclusion

The Logistic Regression classifier was successfully implemented on the Breast Cancer dataset. The model achieved high accuracy and F1-scores, demonstrating its effectiveness in probability-based binary classification tasks.

Viva Questions

- Q1.** If Logistic Regression uses a linear equation, why is it categorized as a classification algorithm?
- Q2.** Explain the mathematical purpose of the Sigmoid function in this algorithm.
- Q3.** What is a decision threshold, and in what scenarios might you want to change it from the default 0.5?
- Q4.** What is the core difference between binary and multiclass classification?
- Q5.** Why is feature scaling (like Standardization) highly recommended before training a Logistic Regression model?

Assignment / Practice Problems

1. Build a Logistic Regression model for a custom student dataset to predict pass/fail based on study hours and attendance.
2. Modify the decision threshold explicitly in Python to 0.3, 0.5, and 0.7. Compare how the Precision and Recall metrics change.
3. Plot the Receiver Operating Characteristic (ROC) curve and calculate the Area Under the Curve (AUC) score.

Experiment 6: Decision Tree Classification

Objective

- Implement Decision Tree classification on a multi-class dataset.
- Understand tree-based algorithmic decision-making and recursive splitting.
- Generate a visual representation of the trained Decision Tree.
- Study the effect of hyperparameter tuning, specifically maximum tree depth.

Theory

A Decision Tree is a non-parametric supervised learning algorithm that recursively divides data using feature-based rules (if-else conditions).

The model evaluates features to find the "best" split that separates classes most effectively. Typical splitting mathematical criteria include:

- **Gini Impurity:** Measures the probability of misclassifying a randomly chosen element. Lower is better.
- **Entropy & Information Gain:** Measures the level of impurity or randomness in a node. The tree maximizes information gain.

A tree continues splitting data into sub-nodes until a stopping condition (like `max_depth` or zero impurity) is reached.

Course Outcome (CO) Mapping

CO1	CO2	CO3
-----	-----	-----

Algorithm / Steps

Step 1. Load the standard Iris dataset.

Step 2. Split the data into training and testing sets with class stratification.

Step 3. Initialize a `DecisionTreeClassifier` using the Gini criterion and a fixed maximum depth.

Step 4. Train the tree model on the training data.

Step 5. Predict the class labels for the testing data.

Step 6. Calculate the accuracy of the predictions.

Step 7. Use Matplotlib and `plot_tree` to render a visual map of the decision logic.

Python Program

Source Code

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score

# 1. Load Data
data = load_iris()
X = data.data
y = data.target

# 2. Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 3. Model Initialization (with max_depth to prevent overfitting)
model = DecisionTreeClassifier(
    criterion="gini",
    max_depth=3,
    random_state=42
)

# 4. Train Model
model.fit(X_train, y_train)

# 5. Predict & Evaluate
y_pred = model.predict(X_test)
print(f"Accuracy Score: {accuracy_score(y_test, y_pred):.4f}")

# 6. Visualize the Tree
plt.figure(figsize=(14, 8))
plot_tree(
    model,
    feature_names=data.feature_names,
    class_names=data.target_names,
    filled=True,
    rounded=True,
    fontsize=10
)
plt.title("Decision Tree Visualization (Max Depth = 3)")
plt.show()
```

Sample Output

Console Output

Accuracy Score: 0.9333

Note: The structural visualization of the Decision Tree will render in a Matplotlib pop-up window. Students should save the generated image and attach it to their lab record.

Result / Conclusion

A Decision Tree classifier was successfully trained and evaluated on the Iris dataset. Restricting the `max_depth` effectively simplified the tree logic, and the structural visualization successfully demonstrated how feature-based splits are mathematically determined.

Viva Questions

- Q1. Define Information Entropy in the context of Decision Trees.
- Q2. What does Gini Impurity measure, and what does a Gini score of 0 represent?
- Q3. Why are Decision Trees highly prone to overfitting the training data?
- Q4. What is "pruning" in decision tree algorithms?
- Q5. How does the `max_depth` hyperparameter help regulate model complexity?

Assignment / Practice Problems

1. Train multiple Decision Trees on the same data using depths of 2, 3, 5, and None. Compare their testing accuracies.
2. Modify the code to use the `criterion="entropy"` parameter instead of "gini". Observe if the tree structure changes.
3. Plot a line graph mapping training accuracy and testing accuracy versus varying tree depths to visually identify the point of overfitting.

Experiment 7: KNN, Naive Bayes and SVM

Objective

- Implement K-Nearest Neighbours (KNN) classification.
- Implement Gaussian Naive Bayes classification.
- Implement Support Vector Machine (SVM) classification.
- Compare the predictive performance of these different classifiers on a common dataset.

Theory

Different algorithms approach classification using distinct mathematical paradigms:

- **K-Nearest Neighbours (KNN):** A distance-based algorithm that classifies a new sample based on the majority class among its K closest data points in the feature space.
- **Naive Bayes:** A probabilistic classifier based on Bayes' Theorem, relying on the "naive" assumption that all features are independent of each other given the class label.
- **Support Vector Machine (SVM):** Finds an optimal hyperplane in an N -dimensional space that distinctly classifies the data points by maximizing the margin between different classes.

Course Outcome (CO) Mapping

CO1	CO2	CO3
-----	-----	-----

Algorithm / Steps

- Step 1.** Load the Iris dataset from Scikit-learn.
- Step 2.** Split the dataset into training and testing sets with stratification.
- Step 3.** Standardize the feature values using `StandardScaler` (crucial for distance-based models like KNN and SVM).
- Step 4.** Initialize the three models: `KNeighborsClassifier`, `GaussianNB`, and `SVC`.
- Step 5.** Iterate through the models to train them on the scaled training data.
- Step 6.** Predict the class labels for the test set.
- Step 7.** Calculate and print the accuracy for each model to compare results.

Python Program

Source Code

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# 1. Load Data
data = load_iris()
X = data.data
y = data.target

# 2. Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 3. Feature Scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 4. Model Initialization
models = {
    "KNN": KNeighborsClassifier(n_neighbors=5),
    "Naive Bayes": GaussianNB(),
    "SVM": SVC(kernel="rbf", random_state=42)
}

# 5. Training and Evaluation
print("--- Classifier Accuracies ---")
for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    print(f"{name:<15}: {accuracy:.4f}")
```

Sample Output

Console Output

```
--- Classifier Accuracies ---  
KNN           : 0.9667  
Naive Bayes    : 0.9667  
SVM            : 1.0000
```

Result / Conclusion

KNN, Naive Bayes, and SVM classifiers were successfully implemented and trained on the Iris dataset. By comparing their accuracy scores, we observed how different algorithmic approaches perform on the same standardized dataset.

Viva Questions

- Q1.** Why is feature scaling (Standardization) absolutely critical for KNN and SVM, but less important for Naive Bayes?
- Q2.** What does the K represent in the KNN algorithm, and how does choosing a very small vs. very large K affect the model?
- Q3.** What is the "naive" assumption made by the Naive Bayes classifier?
- Q4.** In the context of SVM, what is a "support vector"?
- Q5.** What is the "kernel trick" in SVM, and why is the RBF (Radial Basis Function) kernel widely used?

Assignment / Practice Problems

1. Modify the KNN implementation to loop through K values from 1 to 15. Plot a graph of K vs. Accuracy to find the optimal K .
2. Compare the performance of a Linear SVM (`kernel="linear"`) versus an RBF SVM on a non-linear dataset.
3. Extend the code to calculate and print the Precision, Recall, and F1-score for all three classifiers.

Experiment 8: K-Means Clustering

Objective

- Implement K-Means clustering, an unsupervised learning algorithm.
- Understand centroid-based partitioning of data.
- Utilize the Elbow method to mathematically estimate the optimal number of clusters.
- Visualize the resulting clusters and their boundaries.

Theory

K-Means is an unsupervised algorithm that partitions unlabeled data into K distinct, non-overlapping subgroups (clusters).

The algorithm iteratively performs two main steps:

1. **Assignment:** Assigns each data point to the nearest cluster centroid (based on Euclidean distance).
2. **Update:** Recalculates the centroids as the mean of all points assigned to that cluster.

This repeats until convergence. The objective is to minimize **Inertia** (Within-Cluster Sum of Squares - WCSS). The **Elbow Method** helps find the optimal K by plotting Inertia against K and finding the "elbow" point where the rate of decrease sharply shifts.

Course Outcome (CO) Mapping

CO1	CO2	CO3
-----	-----	-----

Algorithm / Steps

Step 1. Load the Wine dataset and select two features to allow for 2D visualization.

Step 2. Scale the features using `StandardScaler`.

Step 3. Run a loop to apply K-Means for $K = 1$ through 8.

Step 4. Store the model's `inertia_` for each K .

Step 5. Plot the Elbow curve (Inertia vs. K) to identify the optimal number of clusters.

Step 6. Train a final K-Means model using the identified optimal K .

Step 7. Generate a scatter plot coloring the data points based on their assigned cluster labels.

Python Program

Source Code

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

# 1. Load Data
data = load_wine()
# Select specific features for 2D visualization (Alcohol & Color Intensity)
X = data.data[:, [0, 9]]

# 2. Feature Scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 3. Elbow Method to find optimal K
inertias = []
K_range = range(1, 9)

for k in K_range:
    model = KMeans(n_clusters=k, random_state=42, n_init=10)
    model.fit(X_scaled)
    inertias.append(model.inertia_)

plt.figure(figsize=(7, 4))
plt.plot(K_range, inertias, marker="o", linestyle="--", color="b")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Inertia (WCSS)")
plt.title("Elbow Method for Optimal K")
plt.grid(True)
plt.show()

# 4. Final Model with optimal K (e.g., K=3)
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
labels = kmeans.fit_predict(X_scaled)

# 5. Visualizing the Clusters
plt.figure(figsize=(7, 5))
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=labels, cmap="viridis", edgecolor="k")
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1],
            s=200, c='red', marker='X', label='Centroids')
plt.xlabel(data.feature_names[0])
plt.ylabel(data.feature_names[9])
plt.title("K-Means Clustering (K=3)")
plt.legend()
plt.show()
```

Sample Output

Graphical Output Note

This unsupervised algorithm relies heavily on visual interpretation. The code will generate two graphical plots:

1. **Elbow Curve:** A line graph showing Inertia dropping sharply and forming an "elbow" at $K = 3$.
2. **Cluster Scatter Plot:** A 2D plot showing data points colored by their 3 respective clusters, with red 'X' markers indicating the mathematical center (centroid) of each cluster.

Note to Student: Attach the screenshots of both generated Matplotlib plots here.

Result / Conclusion

K-Means clustering was successfully implemented. The Elbow method clearly indicated the optimal number of clusters, and the final scatter plot successfully visualized the distinct data partitions and their calculated centroids.

Viva Questions

- Q1. What is the fundamental difference between classification and clustering?
- Q2. What mathematical metric does "Inertia" represent in the context of K-Means?
- Q3. Why is the random initialization of centroids considered a vulnerability in standard K-Means, and how does KMeans++ fix it?
- Q4. Explain how to read an Elbow Method plot to determine the correct K .
- Q5. What happens to the Inertia value if we set K equal to the total number of data points?

Assignment / Practice Problems

1. Apply K-Means clustering to a Mall Customer Segmentation dataset based on 'Annual Income' and 'Spending Score'.
2. Calculate and print the **Silhouette Score** for $K = 2, 3, 4$, and 5 to numerically validate the Elbow method's finding.

Experiment 9: Hierarchical Clustering

Objective

- Implement agglomerative (bottom-up) hierarchical clustering.
- Construct and interpret a dendrogram to visualize data hierarchies.
- Understand different linkage-based distance metrics.
- Compare hierarchical clustering conceptually and visually with K-Means.

Theory

Unlike K-Means, Hierarchical Clustering does not require pre-defining the number of clusters (K).

Agglomerative Clustering is a "bottom-up" approach:

1. It starts by treating every single data point as its own cluster.
2. It iteratively merges the two closest clusters based on a distance metric.
3. It repeats this until all points are merged into a single massive cluster.

This merging process is visualized using a tree-like diagram called a **Dendrogram**. The distance between clusters is defined by **Linkage Criteria** (e.g., Ward minimizes variance, Single minimizes nearest-neighbor distance, Complete minimizes maximum distance).

Course Outcome (CO) Mapping

CO1	CO2	CO3
-----	-----	-----

Algorithm / Steps

Step 1. Load the standard Iris dataset and isolate two features for 2D plotting.

Step 2. Standardize the data using `StandardScaler`.

Step 3. Use SciPy's `linkage` matrix to compute the distances between clusters using the "ward" method.

Step 4. Plot the hierarchical structure using SciPy's `dendrogram`.

Step 5. Analyze the dendrogram to visually determine a logical threshold cut, deciding the final number of clusters.

Step 6. Apply Scikit-learn's `AgglomerativeClustering` to formally group the data points.

Step 7. Plot the clustered data on a scatter plot.

Python Program

Source Code

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage

# 1. Load Data
data = load_iris()
# Selecting Sepal Length and Petal Length
X = data.data[:, [0, 2]]

# 2. Feature Scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 3. Generate Dendrogram using SciPy
Z = linkage(X_scaled, method="ward")

plt.figure(figsize=(10, 6))
dendrogram(Z)
plt.title("Hierarchical Clustering Dendrogram (Ward Linkage)")
plt.xlabel("Data Points (Indices)")
plt.ylabel("Euclidean Distance")
plt.axhline(y=10, color='r', linestyle='--') # Example cut-off line
plt.show()

# 4. Agglomerative Clustering using Scikit-Learn
model = AgglomerativeClustering(n_clusters=3, linkage="ward")
labels = model.fit_predict(X_scaled)

# 5. Visualize Results
plt.figure(figsize=(7, 5))
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=labels, cmap="viridis", edgecolor="k")
plt.xlabel(data.feature_names[0])
plt.ylabel(data.feature_names[2])
plt.title("Agglomerative Clustering Results (K=3)")
plt.show()
```

Sample Output

Graphical Output Note

The script outputs two visual plots for analysis:

1. **Dendrogram:** A tree-like graph showing the iterative merging of data points. By drawing a horizontal line across the longest vertical, uninterrupted branches, we can deduce the optimal number of clusters (in this case, yielding 3 distinct branches).
2. **Cluster Scatter Plot:** A 2D plot mapping Sepal Length vs. Petal Length, distinctly colored into 3 groupings based on the Agglomerative algorithm.

Note to Student: Attach the screenshots of both generated Matplotlib plots here.

Result / Conclusion

Agglomerative Hierarchical clustering was successfully implemented. The dataset's cluster hierarchy was effectively visualized using a dendrogram, which provided a logical, visual method for selecting the optimal number of clusters without relying on trial-and-error guessing.

Viva Questions

- Q1. What is the core difference between Agglomerative (bottom-up) and Divisive (top-down) hierarchical clustering?
- Q2. How do you read a dendrogram to determine the optimal number of clusters?
- Q3. What is meant by "Linkage Criteria"? Name two types other than Ward.
- Q4. What is the specific objective of "Ward's Method" linkage?
- Q5. Name one major advantage and one major disadvantage of Hierarchical Clustering compared to K-Means.

Assignment / Practice Problems

1. Generate and compare three separate dendrograms using `method="single"`, `method="complete"`, and `method="ward"`. Observe how the cluster structures change.
2. Implement hierarchical clustering on a larger dataset and note the execution time compared to K-Means to observe computational complexity.

Experiment 10: Principal Component Analysis (PCA)

Objective

- Understand the mathematical concept of dimensionality reduction.
- Implement Principal Component Analysis (PCA) using Scikit-learn.
- Reduce a high-dimensional dataset to two dimensions while preserving maximum variance.
- Visualize the transformed data points in a 2D feature space.

Theory

Principal Component Analysis (PCA) is an unsupervised statistical technique that transforms high-dimensional data containing correlated features into a smaller set of linearly uncorrelated variables called **Principal Components**.

These components are ordered by the amount of original variance they explain. The first principal component captures the maximum possible variance, the second captures the maximum remaining variance, and so on.

PCA is widely used for:

- **Visualization:** Projecting 3D+ data down to 2D or 3D plots.
- **Data Compression:** Reducing storage space and memory footprint.
- **Noise Reduction:** Filtering out less important features.
- **Speeding up Algorithms:** Reducing computational complexity for downstream ML models.

Course Outcome (CO) Mapping

CO1	CO2	CO3
-----	-----	-----

Algorithm / Steps

Step 1. Load the 13-dimensional Wine dataset from Scikit-learn.

Step 2. Standardize all features to have a mean of 0 and a variance of 1 (Crucial for PCA).

Step 3. Initialize a PCA object specifying `n_components=2`.

Step 4. Fit the PCA model to the scaled data and transform the data into the new lower-dimensional space.

Step 5. Extract and print the explained variance ratio to see how much information was retained.

Step 6. Plot the transformed observations on a 2D scatter plot, coloring them by their target class.

Python Program

Source Code

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# 1. Load Data
data = load_wine()
X = data.data
y = data.target

# 2. Standardization
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 3. Apply PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# 4. Display Dimensionality and Variance Info
print(f"Original Dimensions : {X.shape}")
print(f"Reduced Dimensions : {X_pca.shape}")

print("\n--- Explained Variance Ratio ---")
print(pca.explained_variance_ratio_)
total_variance = sum(pca.explained_variance_ratio_) * 100
print(f"Total Variance Explained by 2 Components: {total_variance:.2f}%")

# 5. Visualization
plt.figure(figsize=(8, 6))
scatter = plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y, cmap="viridis", edgecolor="k"
)

plt.xlabel(f"Principal Component 1 ({pca.explained_variance_ratio_[0]*100:.1f}%)")
plt.ylabel(f"Principal Component 2 ({pca.explained_variance_ratio_[1]*100:.1f}%)")
plt.title("PCA Projection of Wine Dataset")

# Add a legend using the target names
handles, _ = scatter.legend_elements()
plt.legend(handles, data.target_names, title="Classes")
plt.grid(True, alpha=0.5)
plt.show()
```

Sample Output

Console Output

```
Original Dimensions : (178, 13)
Reduced Dimensions  : (178, 2)
```

```
--- Explained Variance Ratio ---
[0.36198848 0.1920749 ]
```

```
Total Variance Explained by 2 Components: 55.41%
```

Note: The scatter plot demonstrating the 3 wine classes distinctly separated along the 2 Principal Components will render in a separate Matplotlib window. Students should attach the plot screenshot here.

Result / Conclusion

PCA was successfully applied to the Wine dataset, reducing its dimensionality from 13 features to just 2 while retaining over 55% of the dataset's variance. The 2D visualization confirmed that the principal components successfully captured the variance needed to linearly separate the different wine classes.

Viva Questions

- Q1. What is the mathematical meaning of "dimensionality reduction"?
- Q2. Why must data be standardized (mean=0, variance=1) before applying PCA?
- Q3. What does the "explained variance ratio" tell us about a principal component?
- Q4. Is PCA considered a supervised or unsupervised learning technique? Why?
- Q5. Does applying PCA always improve the accuracy of a classification model? Why or why not?

Assignment / Practice Problems

1. Reduce the Breast Cancer dataset to two principal components and plot the results.
2. Modify the PCA initialization to `PCA(n_components=0.95)` and write a script to print how many components were kept to retain 95% of the variance.
3. Train a Logistic Regression model on the original Wine dataset, then train another on the PCA-reduced dataset. Compare their testing accuracies and execution times.

Experiment 11: Model Evaluation and Cross-Validation

Objective

- Understand standard train-test evaluation methodologies.
- Calculate comprehensive metrics: Accuracy, Precision, Recall, and F1-score.
- Generate and interpret a Confusion Matrix for a binary classifier.
- Implement k -fold cross-validation to rigorously validate model generalization.

Theory

A Confusion Matrix maps actual vs. predicted values to yield True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN).

Key binary classification metrics include:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$\text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Cross-Validation (e.g., k -fold) systematically divides the data into k equal parts (folds). It trains the model on $k - 1$ folds and tests it on the remaining fold, rotating until every fold has been used as a test set. This yields a more statistically reliable estimate of model performance than a single train-test split.

Course Outcome (CO) Mapping

CO1	CO2	CO3
-----	-----	-----

Algorithm / Steps

Step 1. Load the Breast Cancer dataset.

Step 2. Perform an initial train-test split (80-20) for standard metric evaluation.

Step 3. Build a Scikit-learn Pipeline combining a `StandardScaler` and a `LogisticRegression` model.

Step 4. Train the pipeline and predict values for the test set.

Step 5. Calculate and print the Confusion Matrix and a full `classification_report`.

Step 6. Initialize a `StratifiedKFold` object for 5 folds.

Step 7. Execute `cross_val_score` using the full dataset and print the array of fold scores and the overall mean.

Python Program

Source Code

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score,
    StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report
)

# 1. Load Data
data = load_breast_cancer()
X, y = data.data, data.target

# 2. Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 3. Create Pipeline (Scaling + Modeling)
model_pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("classifier", LogisticRegression(max_iter=1000, random_state=42))
])

# 4. Standard Train-Test Evaluation
model_pipeline.fit(X_train, y_train)
y_pred = model_pipeline.predict(X_test)

print("--- Single Split Evaluation Metrics ---")
print(f"Accuracy : {accuracy_score(y_test, y_pred):.4f}")
print(f"Precision : {precision_score(y_test, y_pred):.4f}")
print(f"Recall : {recall_score(y_test, y_pred):.4f}")
print(f"F1 Score : {f1_score(y_test, y_pred):.4f}")
```

```

print("\n--- Confusion Matrix ---")
print(confusion_matrix(y_test, y_pred))

print("\n--- Classification Report ---")
print(classification_report(y_test, y_pred, target_names=data.target_names))

# 5. Cross-Validation Evaluation
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model_pipeline, X, y, cv=cv, scoring="accuracy")

print("\n--- 5-Fold Cross-Validation ---")
print("Scores per fold :", [round(s, 4) for s in scores])
print(f"Mean CV Accuracy: {scores.mean():.4f}")
print(f"Standard Dev : {scores.std():.4f}")

```

Sample Output

Console Output

```

--- Single Split Evaluation Metrics ---
Accuracy   : 0.9737
Precision   : 0.9726
Recall      : 0.9861
F1 Score    : 0.9793

--- Confusion Matrix ---
[[40  2]
 [ 1 71]]

--- Classification Report ---
              precision    recall  f1-score   support

malignant      0.98        0.95        0.96         42
benign         0.97        0.99        0.98         72

accuracy              0.97              114
macro avg      0.97        0.97        0.97         114
weighted avg   0.97        0.97        0.97         114

--- 5-Fold Cross-Validation ---
Scores per fold : [0.9825, 0.9737, 0.9912, 0.9737, 0.9823]
Mean CV Accuracy: 0.9807
Standard Dev     : 0.0066

```

Result / Conclusion

The classification model was comprehensively evaluated. The Confusion Matrix and classification report provided detailed insight into class-specific performance. Furthermore, the 5-fold cross-validation confirmed that the model is stable and generalizes well to unseen data, yielding a reliable mean accuracy with low variance.

Viva Questions

- Q1.** Explain the components of a Confusion Matrix (TP , TN , FP , FN).
- Q2.** Give an example of a real-world scenario where Recall is significantly more critical than Precision (e.g., medical diagnostics).
- Q3.** In what scenario can measuring model success by "Accuracy" alone be highly misleading?
- Q4.** What is the purpose of `StratifiedKFold` compared to standard `KFold`?
- Q5.** What is the advantage of reporting the Standard Deviation of cross-validation scores?

Assignment / Practice Problems

1. Given a hypothetical confusion matrix: $\begin{bmatrix} 50 & 10 \\ 5 & 100 \end{bmatrix}$, manually calculate the Accuracy, Precision, Recall, and F1-score.
2. Modify the code to compare 5-fold and 10-fold cross-validation results. Note any differences in computation time and mean accuracy.
3. Write a script to evaluate a KNN classifier and an SVM classifier using the identical `StratifiedKFold` CV strategy.

Experiment 12: Ensemble Learning

Objective

- Understand the statistical advantage of ensemble learning techniques.
- Implement a **Random Forest** classifier (a Bagging ensemble method).
- Implement an **AdaBoost** classifier (a Boosting ensemble method).
- Empirically compare ensemble performance metrics against a single standard Decision Tree.

Theory

Ensemble learning mathematically aggregates the predictions of multiple diverse models (often weak learners) to produce a single, robust final prediction.

Multiple Base Learners → Strong Combined Prediction

- **Random Forest (Bagging):** Builds numerous independent Decision Trees on random subsets of data and features. The final prediction is determined by a majority vote. This heavily reduces model variance (overfitting).
- **AdaBoost (Boosting):** Trains multiple weak learners (usually depth-1 trees, or "stumps") sequentially. Each new tree pays strictly more attention to the observations that the previous tree misclassified. This sequentially reduces model bias.

Course Outcome (CO) Mapping

CO1	CO2	CO3
-----	-----	-----

Algorithm / Steps

Step 1. Load the Wine classification dataset.

Step 2. Split the data into stratified training and testing sets.

Step 3. Initialize a single `DecisionTreeClassifier`.

Step 4. Initialize a `RandomForestClassifier` specifying `n_estimators=100` (100 trees).

Step 5. Initialize an `AdaBoostClassifier` specifying `n_estimators=100`.

Step 6. Loop through a Python dictionary containing all three models.

Step 7. Fit each model, generate predictions, and print the resulting test set accuracy.

Python Program

Source Code

```
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier
from sklearn.metrics import accuracy_score

# 1. Load Data
data = load_wine()
X, y = data.data, data.target

# 2. Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 3. Define Models
models = {
    "Decision Tree": DecisionTreeClassifier(
        max_depth=4,
        random_state=42
    ),
    "Random Forest": RandomForestClassifier(
        n_estimators=100,
        random_state=42
    ),
    "AdaBoost": AdaBoostClassifier(
        n_estimators=100,
        algorithm="SAMME", # Prevents deprecation warnings in newer sklearn
                           versions
        random_state=42
    )
}

# 4. Train, Predict, and Evaluate
print("--- Model Accuracy Comparison ---")
for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    score = accuracy_score(y_test, y_pred)

    # Format output for alignment
    print(f"{name:<15}: {score:.4f}")
```

Sample Output

Console Output

```
--- Model Accuracy Comparison ---  
Decision Tree   : 0.9444  
Random Forest   : 1.0000  
AdaBoost        : 0.9167
```

Result / Conclusion

The Decision Tree, Random Forest, and AdaBoost models were successfully trained and evaluated. The comparison empirically demonstrated that Ensemble models (like Random Forest) often outperform single, isolated learners (Decision Trees) by reducing variance and increasing overall prediction robustness.

Viva Questions

- Q1.** Explain the fundamental logic behind Ensemble Learning.
- Q2.** What is the main difference between Bagging (e.g., Random Forest) and Boosting (e.g., AdaBoost)?
- Q3.** Why does a Random Forest purposefully use only a random subset of features for each split?
- Q4.** What is a "weak learner," and why are they specifically utilized in AdaBoost?
- Q5.** Can an ensemble model still overfit the training data?

Assignment / Practice Problems

1. Write a loop to train a Random Forest model with `n_estimators` set to 10, 50, 100, 200, and 500. Plot how the number of trees affects testing accuracy.
2. Compare Random Forest and AdaBoost performance on a much more complex dataset using the F1-score metric instead of pure Accuracy.
3. Implement a `GradientBoostingClassifier` and add it to the comparison dictionary.

Experiment 13: Hyperparameter Tuning using Grid Search

Objective

- Understand the difference between model parameters and hyperparameters.
- Use `GridSearchCV` to systematically evaluate multiple hyperparameter combinations.
- Select the optimal configuration based on cross-validation performance.
- Combine preprocessing pipelines with hyperparameter tuning to prevent data leakage.

Theory

Hyperparameters are configuration variables whose values are set *before* the learning process begins (e.g., learning rate, K in KNN, max depth of a tree, or SVM kernel). They control the learning process itself, unlike normal parameters (like weights) which are derived via training.

Grid Search is an exhaustive search algorithm. It builds a model for every possible combination of hyperparameters specified in a predefined grid, evaluates them using cross-validation, and returns the best-performing model:

Parameter Grid $\rightarrow$ Cross-Validation $\rightarrow$ Best Parameters

Course Outcome (CO) Mapping

CO1	CO2	CO3
-----	-----	-----

Algorithm / Steps

- Step 1.** Load the Iris dataset and split it into stratified training and testing sets.
- Step 2.** Create a Pipeline that first standardizes the data and then applies an SVM classifier (SVC).
- Step 3.** Define a hyperparameter dictionary (`param_grid`) specifying the kernels, C values, and γ values to test.
- Step 4.** Initialize `GridSearchCV` using the pipeline, parameter grid, and 5-fold cross-validation.
- Step 5.** Fit the grid search object to the training data.
- Step 6.** Extract and print the best hyperparameter combination and its corresponding CV score.
- Step 7.** Evaluate the automatically selected best estimator on the unseen test data.

Python Program

Source Code

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# 1. Load Data
data = load_iris()
X, y = data.data, data.target

# 2. Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 3. Create Pipeline
pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("svm", SVC(random_state=42))
])

# 4. Define Hyperparameter Grid
# Note the "svm_" prefix connects parameters to the pipeline step named "svm"
param_grid = {
    "svm__kernel": ["linear", "rbf"],
    "svm__C": [0.1, 1, 10],
    "svm__gamma": ["scale", 0.01, 0.1]
}

# 5. Initialize & Run Grid Search
grid = GridSearchCV(
    pipeline,
    param_grid,
    cv=5,
    scoring="accuracy",
    n_jobs=-1
)

grid.fit(X_train, y_train)

# 6. Display Results
print("--- Grid Search Results ---")
print(f"Best Parameters: {grid.best_params_}")
print(f"Best CV Score : {grid.best_score_:.4f}")
```

```
# 7. Evaluate on Test Set
best_model = grid.best_estimator_
y_pred = best_model.predict(X_test)

print(f"\nTest Accuracy : {accuracy_score(y_test, y_pred):.4f}")
```

Sample Output

Console Output

```
--- Grid Search Results ---
Best Parameters: {'svm__C': 1, 'svm__gamma': 'scale', 'svm__kernel': 'linear'}
Best CV Score   : 0.9750

Test Accuracy   : 0.9667
```

Result / Conclusion

Grid Search was successfully implemented to tune the Support Vector Machine pipeline. By exhaustively combining and validating hyperparameters across 5 folds, the algorithm successfully identified the optimal configuration (C=1, linear kernel) that maximized accuracy while preventing overfitting.

Viva Questions

- Q1. Explain the distinct difference between a model *parameter* (like a weight) and a *hyper-parameter* (like learning rate).
- Q2. Why must we prefix parameters with `svm__` in the `param_grid` dictionary when using a Pipeline?
- Q3. Why is Cross-Validation strictly necessary during hyperparameter tuning (i.e., why not just evaluate on a single test set)?
- Q4. What is the computational disadvantage of using `GridSearchCV` over `RandomizedSearchCV` when searching a very large grid?
- Q5. What does the `n_jobs=-1` argument do?

Assignment / Practice Problems

1. Perform Grid Search on a KNN classifier using different values of K (e.g., 1 to 20) and distance metrics (`weights="uniform"` vs `"distance"`).

2. Tune a Decision Tree classifier using a grid testing `max_depth`, `min_samples_split`, and `criterion` (gini vs. entropy).
3. Implement `RandomizedSearchCV` on the same SVM pipeline and compare the execution time and final accuracy against Grid Search.

Experiment 14: Introduction to Neural Networks

Objective

- Understand the fundamental architecture of an Artificial Neural Network (ANN).
- Implement a multi-layer feed-forward neural network using Scikit-learn's `MLPClassifier`.
- Train the network on a complex, higher-dimensional dataset (Digits).
- Evaluate neural network classification performance via accuracy and confusion matrices.

Theory

An **Artificial Neural Network** consists of interconnected processing units called neurons, arranged in layers.

A single neuron mathematically computes a weighted sum of its inputs plus a bias:

$$z = \sum_{i=1}^n w_i x_i + b$$

The result z is then passed through a non-linear **Activation Function** (e.g., ReLU, Sigmoid, or Tanh) to allow the network to learn complex patterns.

A standard Feed-Forward Neural Network (Multi-Layer Perceptron) flows in one direction:

Input Layer → Hidden Layer(s) → Output Layer

Course Outcome (CO) Mapping

CO1	CO2	CO3
-----	-----	-----

Algorithm / Steps

- Step 1.** Load the Digits dataset (images of handwritten numbers 0-9).
- Step 2.** Split the data into stratified training and testing subsets (80% / 20%).
- Step 3.** Standardize the input features (crucial for neural network convergence).
- Step 4.** Initialize an `MLPClassifier` with two hidden layers (64 and 32 neurons) and the ReLU activation function.
- Step 5.** Train (fit) the neural network using backpropagation over the training data.
- Step 6.** Predict the output labels for the testing data.
- Step 7.** Calculate the overall accuracy and print a Confusion Matrix to evaluate per-digit classification performance.

Python Program

Source Code

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
import numpy as np

# 1. Load Data
data = load_digits()
X, y = data.data, data.target

# 2. Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 3. Feature Scaling (Highly recommended for NNs)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 4. Initialize Multi-Layer Perceptron (Neural Network)
model = MLPClassifier(
    hidden_layer_sizes=(64, 32), # 2 hidden layers: 64 neurons, then 32 neurons
    activation="relu",
    solver="adam",
    max_iter=500,
    random_state=42
)

# 5. Train Model
model.fit(X_train, y_train)

# 6. Predict and Evaluate
y_pred = model.predict(X_test)

print("--- Neural Network Performance ---")
print(f"Test Accuracy : {accuracy_score(y_test, y_pred):.4f}")

print("\n--- Confusion Matrix ---")
print(confusion_matrix(y_test, y_pred))
```

Sample Output

Console Output

```
--- Neural Network Performance ---
```

```
Test Accuracy : 0.9833
```

```
--- Confusion Matrix ---
```

```
[[36  0  0  0  0  0  0  0  0  0]
 [ 0 35  0  0  0  0  0  0  1  0]
 [ 0  0 35  0  0  0  0  0  0  0]
 [ 0  0  0 37  0  0  0  0  0  0]
 [ 0  0  0  0 36  0  0  0  0  0]
 [ 0  0  0  0  0 36  0  0  0  0]
 [ 0  0  0  0  0  0 36  0  0  0]
 [ 0  0  0  0  0  0  0 36  0  0]
 [ 0  1  0  0  0  0  0  0 33  0]
 [ 0  0  0  1  0  0  0  1  2 32]]
```

Result / Conclusion

A feed-forward Multi-Layer Perceptron (MLP) neural network was successfully trained on the 64-dimensional Digits dataset. By utilizing two hidden layers and the ReLU activation function, the network captured complex non-linear patterns, achieving an accuracy exceeding 98% in recognizing handwritten digits.

Viva Questions

- Q1.** Explain the mathematical operation that occurs inside a single artificial neuron.
- Q2.** Why is an "Activation Function" absolutely necessary in a neural network? (What happens if we remove it?)
- Q3.** Why is ReLU (Rectified Linear Unit) currently the most widely used activation function for hidden layers over older functions like Sigmoid?
- Q4.** What is backpropagation, and how does it relate to adjusting the network's weights?
- Q5.** Why is standardizing/scaling input features critical before passing data into a neural network?

Assignment / Practice Problems

1. Train an MLP with only one hidden layer (`hidden_layer_sizes=(32,)`) and compare its accuracy to the two-layer network.

2. Modify the code to compare the `activation="relu"` function against `activation="tanh"`.
3. Extract the loss curve (`model.loss_curve_`) and plot the training loss versus the number of iterations using Matplotlib.

Experiment 15: Mini-Project / Integrated Case Study

Objective

- Integrate the complete Machine Learning workflow into a single end-to-end application.
- Perform independent data preparation and exploratory data analysis (EDA).
- Implement and tune multiple Machine Learning algorithms.
- Critically compare model performance using appropriate evaluation metrics.
- Interpret, document, and communicate experimental results professionally.

Theory

A standard Machine Learning project lifecycle generally follows a structured pipeline:

Problem Definition → Data Collection → Preprocessing
 ↪ Model Building → Evaluation → Comparison → Interpretation

For this integrated case study, students must select a meaningful dataset and logically justify:

- **Significance:** Why the problem matters and what impact the predictions have.
- **Feature Engineering:** Which features are retained, modified, or dropped.
- **Algorithm Choice:** Why specific ML models were selected for the data type.
- **Metrics:** Which evaluation metrics (e.g., Recall over Accuracy) fit the business logic.
- **Conclusions:** Why one model empirically outperformed the others.

Course Outcome (CO) Mapping

CO1	CO2	CO3	CO4
-----	-----	-----	-----

Mini-Project Requirements

Each student group must complete the following stages systematically:

Step 1. Select a real-world problem statement.

Step 2. Identify and download a suitable dataset (e.g., from Kaggle, UCI).

Step 3. Clearly define the target variable and independent features.

Step 4. Perform comprehensive Exploratory Data Analysis (EDA) with visualizations.

Step 5. Clean and preprocess the data (handling nulls, scaling, encoding).

Step 6. Split the dataset appropriately (Train/Test or Cross-Validation).

Step 7. Implement a minimum of **three** distinct ML models.

Step 8. Evaluate the models using multiple appropriate metrics.

Step 9. Perform a comparative analysis of the results.

Step 10. Select the final, best-performing model with concrete justification.

Step 11. Prepare visual graphs summarizing important findings and model comparisons.

Step 12. Prepare a concise, professionally formatted technical report.

Step 13. Present and demonstrate the project during the final lab evaluation.

Suggested Project Report Structure

1. Title Page & Abstract
2. Problem Statement & Objectives
3. Dataset Description (Source, Size, Attributes)
4. Exploratory Data Analysis (EDA)
5. Data Preprocessing & Feature Engineering
6. Model Selection & Methodology
7. Training Phase (including Hyperparameter Tuning)
8. Evaluation Metrics
9. Comparative Analysis (Tables & Graphs)
10. Results, Discussion, and Conclusion
11. Future Scope

Illustrative Integrated Program

Source Code (Reference Framework)

```
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score,
    f1_score

# 1. Load Dataset
data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = pd.Series(data.target, name="target")

print(f"Dataset Shape: {X.shape}\n")

# 2. Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 3. Define Models inside Pipelines
models = {
    "Logistic Regression": Pipeline([
        ("scaler", StandardScaler()),
        ("model", LogisticRegression(max_iter=1000, random_state=42))
    ]),
    "KNN": Pipeline([
        ("scaler", StandardScaler()),
        ("model", KNeighborsClassifier(n_neighbors=5))
    ]),
    "Random Forest": RandomForestClassifier(
        n_estimators=100, random_state=42
    ) # Tree models don't strictly require scaling
}

# 4. Train, Predict, and Evaluate
results = []

for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
```

```

    results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(y_test, y_pred),
        "Recall": recall_score(y_test, y_pred),
        "F1-Score": f1_score(y_test, y_pred)
    })

# 5. Compile and Display Results
results_df = pd.DataFrame(results).round(4)
print("--- Final Model Comparison ---")
print(results_df.to_string(index=False))

```

Expected Analysis Format

Students must synthesize their findings into a standardized comparison table:

Model Comparison Table

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	–	–	–	–
K-Nearest Neighbours	–	–	–	–
Random Forest	–	–	–	–

Note: Actual values will strictly depend on the dataset, preprocessing strategy, cross-validation parameters, and model configuration chosen by the student.

Result / Conclusion

An end-to-end Machine Learning workflow was successfully implemented. This included data acquisition, preprocessing, iterative model training, multi-metric evaluation, and comparative analysis. The optimal model was successfully identified based on empirical evidence and problem constraints.

Project Viva Questions

- Q1. What is the primary real-world motivation behind choosing this specific dataset?
- Q2. Explain the distribution of your target variable. Was there a class imbalance?
- Q3. Why did you select these particular ML algorithms over others?
- Q4. Detail the exact preprocessing operations you performed and why they were necessary.

- Q5.** Based on your problem domain, which evaluation metric is the most critical and why?
- Q6.** Did any of your models suffer from overfitting? How did you detect and mitigate it?
- Q7.** How did you ensure your model comparison was completely fair and unbiased?
- Q8.** Which model performed the best overall, and what algorithmic traits allowed it to do so?
- Q9.** What are the current limitations of your deployed model?
- Q10.** How could this machine learning system be improved in a future iteration?

Mini-Project Deliverables

On the day of evaluation, each group must submit/present:

1. Fully executable Source Code (e.g., a commented Jupyter Notebook).
2. The original Dataset (or a reproducible link to the source).
3. A comprehensive Technical Report following the suggested structure.
4. High-quality graphs (e.g., Correlation Heatmaps, ROC curves, bar charts).
5. Presentation slides (5-10 slides summarizing the workflow).
6. A short live demonstration of the code execution.

Extended Machine Learning Project Ideas

Suggested Capstone / Case Study Topics

Students may choose from the following application areas or propose their own equivalent topic.

- 1. Student Performance Prediction:** Predict final grades or dropout risks using attendance, internal marks, and demographic features.
- 2. House Price Prediction (Regression):** Build predictive models to estimate real-estate prices based on location, square footage, and amenities.
- 3. Customer Churn Prediction:** Classify whether a telecom or SaaS customer is likely to cancel their subscription within the next quarter.
- 4. Loan Approval Classification:** Automate the classification of banking loan applications using applicant credit history and financial features.
- 5. Credit Card Fraud Detection:** Identify fraudulent transactions in a highly imbalanced financial dataset.
- 6. Diabetes Risk Prediction:** Build diagnostic classification models using patient medical attributes (e.g., BMI, insulin, age).
- 7. Heart Disease Prediction:** Compare multiple classification models (SVM, Random Forest, KNN) for predicting cardiovascular disease risk.
- 8. Customer Segmentation (Unsupervised):** Use K-Means and Hierarchical Clustering to discover distinct purchasing groups in retail data.
- 9. Wine Quality Prediction:** Predict the quality rating of wines using chemical properties via regression or multi-class classification.
- 10. Employee Attrition Prediction:** Predict employee turnover (resignation risk) using organizational, survey, and salary features.
- 11. Crop Yield Prediction:** Use agricultural, soil, and environmental variables (rainfall, temperature) to forecast crop output.
- 12. Air Quality Index (AQI) Forecasting:** Predict pollution levels using historical weather and emissions data.
- 13. Spam Email Classification:** Utilize Natural Language Processing (NLP) basics and Naive Bayes to classify messages as spam or legitimate.
- 14. Sentiment Analysis on Reviews:** Classify IMDB or Amazon text reviews as positive, negative, or neutral.

15. **Movie Recommendation System:** Develop a collaborative filtering or content-based recommendation system using user-item interaction data.
16. **Basic Image Classification:** Train an MLP or simple CNN on a standard dataset (like MNIST or CIFAR-10) to recognize objects or digits.
17. **Predictive Maintenance for Manufacturing:** Predict whether an industrial machine or component is nearing failure and requires maintenance.
18. **Traffic Volume Prediction:** Forecast highway traffic volume using historical temporal and weather-related data.
19. **Energy Demand Forecasting:** Predict city-wide electricity consumption using time-series and environmental features.
20. **Network Intrusion / Anomaly Detection:** Identify unusual or malicious network traffic observations using unsupervised learning techniques.

1. **Predictive Maintenance with Sensor Fusion:** Predict industrial machine failure time-to-event (Remaining Useful Life) using multivariate time-series data from IoT sensors and deep learning (e.g., LSTMs).
2. **Explainable AI (XAI) in Medical Diagnosis:** Predict diabetic retinopathy or heart disease risk and utilize SHAP/LIME frameworks to provide clinician-friendly, interpretable explanations for the model's predictions.
3. **Credit Card Fraud Detection via Cost-Sensitive Learning:** Identify fraudulent transactions in highly imbalanced financial datasets using SMOTE, Isolation Forests, and cost-sensitive neural networks.
4. **Customer Churn & Survival Analysis:** Move beyond binary classification by using survival analysis (e.g., Cox Proportional Hazards) to predict *when* a SaaS customer will churn and calculate their Lifetime Value (CLV).
5. **Multivariate Crop Yield & Disease Forecasting:** Combine satellite imagery (NDVI), soil sensors, and meteorological time-series data to predict agricultural yield and detect early signs of crop blight.
6. **Spatiotemporal Air Quality (AQI) Forecasting:** Use advanced time-series models (Prophet or Graph Neural Networks) to forecast pollution levels across multiple city nodes using historical emissions and traffic data.
7. **DeepFake Detection in Audio/Visual Media:** Train a Convolutional Neural Network (CNN) pipeline to distinguish between authentic and AI-generated media by analyzing frequency artifacts and facial inconsistencies.

8. **LLM Prompt Injection Vulnerability Detection:** Build a Natural Language Processing (NLP) classifier to sanitize user inputs and detect prompt injection attacks targeting Large Language Model applications.
9. **Algorithmic Trading & Sentiment Fusion:** Forecast stock price directional movement by fusing traditional historical technical indicators with real-time sentiment analysis of financial news and Twitter feeds.
10. **Dynamic Traffic Routing & Congestion Prediction:** Predict highway traffic density using graph-based models and weather data to optimize real-time routing algorithms for logistics fleets.
11. **Smart Grid Energy Peak Load Forecasting:** Forecast city-wide power consumption using XGBoost or Recurrent Neural Networks to assist utility companies in dynamic grid load balancing.
12. **Zero-Day Network Intrusion Detection:** Identify malicious packet sequences and cyber-attacks using unsupervised Autoencoders and sequential anomaly detection on network traffic logs.
13. **Fine-Grained Emotion Analysis on Text:** Move beyond basic positive/negative sentiment by classifying reviews or social media posts into distinct emotional categories (anger, joy, surprise) using Transformer embeddings.
14. **Context-Aware Hybrid Recommendation Engine:** Build a recommendation system combining collaborative filtering, content-based deep learning, and contextual user signals (time of day, location, device).
15. **Student Dropout Prediction & Intervention Routing:** Predict at-risk students using early-semester behavioral data (LMS logins, assignment latency) and formulate data-driven intervention recommendations.
16. **Real-Estate Valuation with Geospatial Integration:** Predict housing prices by augmenting traditional property features with API-derived geospatial proximity data (distance to transit hubs, schools, and crime heatmaps).
17. **Automated Toxicity & Bias Moderation:** Train a robust NLP model to detect toxic or abusive language in social media while actively measuring and mitigating demographic bias within the model itself.
18. **Real-Time Sign Language Translation:** Use pose-estimation frameworks (like MediaPipe) to extract hand-tracking landmarks and train a classifier to translate complex signs into text.
19. **Retail Supply Chain Demand Forecasting:** Predict product-level inventory requirements across multiple retail locations, accounting for seasonality, promotional events, and macroeconomic indicators.

20. **Medical Image Segmentation (e.g., Tumors/Lesions):** Implement a U-Net architecture to not just classify, but precisely segment MRI/CT scan anomalies at the pixel level.
21. **Autonomous Vehicle Behavioral Cloning:** Train a CNN to predict the continuous steering angle and braking parameters of a vehicle based solely on front-facing dashboard camera footage.
22. **Speech-to-Text for Low-Resource Languages:** Build an Automatic Speech Recognition (ASR) system customized for a specific regional dialect using transfer learning and Mel-Frequency Cepstral Coefficients (MFCC).
23. **Fake News & Disinformation Propagation Graphing:** Classify news articles as fabricated by analyzing linguistic patterns, source credibility, and modeling their propagation through social networks.
24. **Genomic Biomarker Discovery via Dimensionality Reduction:** Apply PCA and t-SNE clustering on high-dimensional genomic data (RNA-seq) to identify cancer subtypes and potential genetic biomarkers.
25. **Wildfire Risk Mapping and Spread Prediction:** Use topographical maps, meteorological data, and historical vegetation dryness indexes to predict the probability and trajectory of forest fires.
26. **AI-Driven Resume Parsing & Semantic Ranking:** Build an NLP pipeline to extract skills from unstructured PDF resumes and rank them against a specific job description using cosine similarity and Word2Vec.
27. **Customer Support Intent Classification:** Train an NLP classifier to analyze support tickets, route queries to appropriate technical departments, and automatically suggest resolution steps.
28. **Music Genre Tagging via Audio Feature Extraction:** Extract acoustic features (chroma frequencies, spectral centroid) from raw audio files to build a model that categorizes music into distinct genres or moods.
29. **Dynamic Pricing Optimization for E-commerce:** Develop a reinforcement learning or regression model to dynamically adjust product prices based on competitor pricing scraping, demand elasticity, and inventory levels.
30. **Early Disease Detection via Keystroke Dynamics:** Use sequence modeling on user typing patterns (key hold time, flight time) to detect early neurodegenerative motor impairments like Parkinson's disease.
31. **Phishing Domain Detection using DOM Analysis:** Combine NLP on URL strings with structural feature analysis of website HTML (DOM trees) to proactively classify zero-day phishing websites.

- 32. Visual Question Answering (VQA):** Build a multimodal model that takes an image (e.g., a medical scan or a street scene) and a text question, outputting a relevant, context-aware text answer.
- 33. Data Center Power Usage Optimization:** Use machine learning to predict server compute loads and optimize HVAC cooling system settings in real-time to minimize Power Usage Effectiveness (PUE).
- 34. Sports Analytics & Injury Prediction:** Analyze historical player statistics, game schedules, and biometric tracking data to predict injury risks and optimize player rest cycles.
- 35. Satellite Imagery Land-Use Classification:** Utilize deep learning on multispectral satellite imagery to classify land use (urban, agriculture, forest, water) for urban planning and deforestation monitoring.

Laboratory Assessment Rubric

Suggested Evaluation Scheme

Evaluation Component	Marks
Regular Lab Performance & Attendance	20
Experiment Implementation & Execution	20
Lab Record / Documentation	10
Viva-Voce / Conceptual Understanding	10
Integrated Mini-Project (Experiment 15)	20
End-Semester Practical Examination	20
Total Maximum Marks	100

Important Assessment Principle

Evaluation should consider not only whether the student's code executes successfully without errors, but critically whether the student can independently:

- **Explain** the underlying mathematical algorithm.
- **Justify** the chosen data preprocessing steps.
- **Interpret** the generated output and visualizations.
- **Select** appropriate evaluation metrics for the specific problem domain.
- **Compare** models critically against one another.
- **Identify** the limitations of their approach and suggest possible improvements.

Recommended Python Environment Setup

Terminal / Command Prompt Installation

```
# 1. Verify Python Installation
python --version

# 2. Install Core Data Science Libraries
pip install numpy
pip install pandas
pip install matplotlib
pip install seaborn
pip install scikit-learn
pip install scipy

# 3. Install Jupyter Environment (Highly Recommended)
pip install jupyter
pip install jupyterlab
```

Suggested Jupyter Notebook Workflow

Step 1. Initialization: Create a new notebook and write a Markdown cell defining the objective.

Step 2. Import Libraries: Group all necessary library imports in the very first code cell.

Step 3. Data Acquisition: Load or define the dataset using Pandas.

Step 4. Exploration (EDA): Explore the data structure, summarize statistics, and check for null values.

Step 5. Preprocessing: Clean the dataset, encode categoricals, and scale features.

Step 6. Modeling: Instantiate and configure the Machine Learning model(s).

Step 7. Training & Prediction: Fit the model to training data and predict on test data.

Step 8. Evaluation: Calculate performance metrics (Accuracy, F1, RMSE, etc.).

Step 9. Visualization: Plot confusion matrices, decision boundaries, or regression lines.

Step 10. Conclusion: Document findings, interpretations, and results in a final Markdown cell.

Machine Learning Lab

Course Code: IT-593

Data → Model → Evaluation → Insight

Department of Information Technology

Dr. B. C. Roy Engineering College, Durgapur

Prepared by Dinesh K. Pradhan, PhD